Bonn, 21. April 2026
Prof. Dr.-Ing. Jan-Henrik Haunert
Dorian Baltzer, M.Sc.

Module Location-Based Services, Summer Semester 2026

Project Task

Deadlines: **Sunday, 17.05.2026**: Task a) - d)
Sunday, 21.06.2026: Task a) - f)
Sunday, 19.07.2026: Full project

In this year's Location-Based Services project, you will develop a Python-based web application that calculates alternative routes for wheelchair navigation using Dijkstra's algorithm. You have already learned the Python basics required for this module in *MGE-06: Profile Fundamentals (GSD)*. Everything else you need to know will be provided to you in the first lectures of this module.
This project is structured with multiple deadlines. For each deadline, you need to upload the full project to eCampus, ensuring that all required tasks are completed. Additionally, the required tasks need to be executable without errors. You will present the results in a final presentation at the end of the semester. The partial grade for the project will include the submitted solutions as well as the final presentation.

Task 1

The alternative routing is based on the paper by Abraham et al. [2013][1]. An initial code base (student_code.zip) is provided to you. This includes a basic setup of `fastapi` and a web visualization. Your focus is to implement the Python backend. The project can be divided into the following subtasks:

a) Familiarize yourself with the code. The shortest walking path is already implemented. For the navigation Dijkstra's algorithm provided by the `NetworkX` package is used. Run the code and visualize the shortest walking path in the provided web interface.

Hints:

- You need to install `fastapi` and all related dependencies preferably in a separate 'anaconda' environment.
- To run the local server open a terminal on the project folder level and run `uvicorn main:app --reload`
- You need to have `anaconda` installed. Then, you can create a new `conda` environment in your terminal using `conda create -n lbs -c conda-forge fastapi networkx pyproj uvicorn osmnx`.

[1] Ittai Abraham, Daniel Delling, Andrew V Goldberg, and Renato F Werneck. 2013. *Alternative routes in road networks*. Journal of Experimental Algorithmics (JEA) 18 (2013), 1–1. https://doi.org/10.1145/2444016.2444019

b) Next, collect and preprocess the data needed to calculate the shortest path for wheelchair navigation. The road data is provided by OpenStreetMaps (OSM). You can download the `OSM` data using *Geofabrik* (`https://download.geofabrik.de/`). Take into account that not all roads can be traversed on foot or by wheelchair. Filter the road data accordingly. To preprocess the data you can either use Python or QGIS. Especially pay attention to the following:

- In a wheelchair it is not possible to move along steps. As such, steps should not be considered for possible routes. The `OSM` road data includes information on steps.
- The difficulty and speed of moving in a wheelchair changes depending on the slope of the road. To account for this, the slope information needs to be added to the directed road edges and the edge weight needs to be adjusted accordingly. Use the digital elevation model to get the heights and calculate the slope for each directed edge. The city of Bonn provides the digital elevation model as open source data (`https://opendata.bonn.de/dataset/digitales-h%C3%B6henmodell-dgm-k%C3%B6ln-bonn`).
- Additionally, not all slopes can be passed using a wheelchair. What is a feasible inclination and why? Implement this into your code.

Hints:

- There are different ways to handle the limitations of wheelchair navigation. As such you can also account for some of these limitations in task c) or d). You are free to choose where and how you address these limitations as long as you handle the limitations of wheelchair navigation before calculating the shortest path for it.

c) From the data you selected, create a `networkX` graph for the navigation task. For this, implement the method `__prepareGraph()` in `GraphHolder.py`. Depending how you processed the data, the steps required to prepare the graph are slightly different.

Hints:

- Pay attention to the tasks formulated in the docstring of `__prepareGraph()`.

d) Now that all data is collecet and preprocessed you can adapt the shortest path calculation for wheelchair navigation. For this, you only need to change the weights used in Dijkstra's algorithm. Implement the method `updateGraphWeights()` in `Util.py`. Once this is done you are able to calculate and visualize the shortest path for wheelchair navigation in the web interface provided to you.

Hints:

- Pay attention to the tasks formulated in the docstring of `updateGraphWeights()`.
- `updateGraphWeights()` is called by `shortest_path_by_wheelchair()` in `Navigation.py`.
- In `_shortest_path()` Dijkstra's algorithm provided by the `networX` library is used. You do not need to change anything in this method.

e) Read the paper *Alternative Routes in Road Networks* by Abraham et al. [2013][1] and take a closer look at the constraints that need to be fulfilled to ensure that the alternative paths are valid and how the via vertices (see paper) are defined. This topic is also discussed in Lecture 4.

f) There are three algorithms described in the paper. In this project you will be implementing a simpler algorithm. Instead of a bidirectional search you can run Dijkstra twice, once starting from source and once starting from target. Implement this in the method `_alternative_paths()` in `Navigation.py`. There is no need to search the full graph for via vertices. As such, think about when and how to terminate Dijkstra's algorithm. The paper also describes the three constraints that a single via path must fulfill. These are:

   (a) $l(Opt \cap P) \leq \gamma \cdot l(Opt)$ (limited sharing);

   (b) P is T-locally optimal for $T = \alpha \cdot l(Opt)$ (local optimality); and

   (c) P is $(1 + \epsilon)$-UBS (uniformly bounded stretch).

   Implement these constraints. Adjust the parameters α, γ and ϵ and explain your choice.

   Hints:

   - Pay attention to the tasks formulated in the docstring of `_alternative_paths()`.
   - Without terminating Dijkstra's algorithm finding possible via nodes might take too long.
   - You need to change the default values for the parameters α, γ and ϵ for the alternative path constraints in the function `get_path()` in `main.py`.

g) Once `_alternative_paths()` is implemented, this should work for both the walking and wheelchair navigation. Test your implementation for both. Adjust the parameters. Explain your parameter choice.

h) Reflect on and discuss your implementation. Where do you see room for improvement?

   - Think of possible ways to visualize the alternative routes. Outline the benefits of your proposed changes and adapt the frontend accordingly.
   - Think about ways to speed up finding alternative paths. Come up with your own heuristic or approximation. Describe your approach and implement this in your code.

   Hints:

   - To not disrupt the implementation for the previous task, create the speedup as an optional add-on. You can make use of an `if-else` statement for this.